bank.ly 帮客力

Technical Documentation

2026 深圳国际金融科技大赛 (FinTechathon)

Artificial Intelligence Track

Team SUSTech 南方科技大学

GitHub repository: github.com/fvia88/ai-banking-agent

1 Executive summary

Banking tasks often require users to navigate separate screens and repeat information. bank.ly brings selected tasks into one conversational interface while retaining explicit authorization for operations that change financial state. It demonstrates three challenge scenarios: smart transfers, spending analysis and subscription management.

Natural-language requests become validated proposals and grounded replies. All accounts, transactions and execution effects are synthetic; no live bank, payment network or merchant cancellation service is connected.

Groq interprets natural language. Our application validates its output, calculates financial values deterministically and keeps execution under backend control. The model never receives authority to execute a transfer or cancellation.

Technical contributions

1. **Grounded intent-to-action pipeline:** typed proposals and correlated tool results separate language understanding from verified financial facts and execution.

2. **Recoverable conversations and dependent tasks:** persistent drafts support missing details, corrections and interruption; a bounded cancellation-then-transfer plan advances only after verified completion.

3. **Guarded simulator execution:** permission tiers, action-bound approval, simulated MFA, persistent safety locks and transactional updates protect state-changing operations.

These contributions are application engineering around existing model APIs; the project does not claim a newly trained model or a novel foundation-model algorithm.

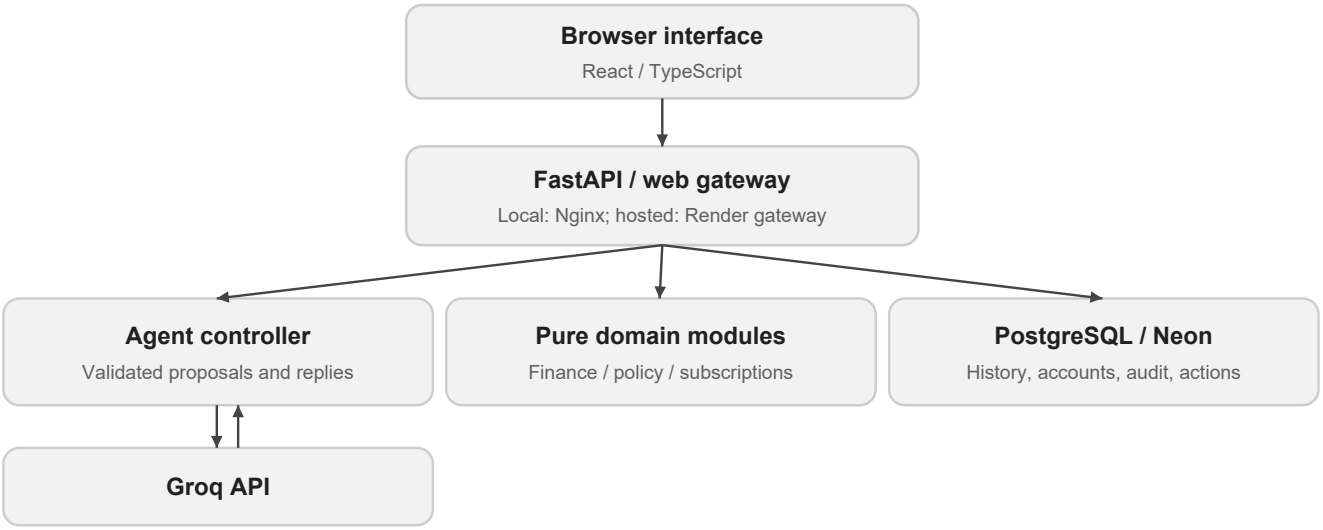
Capability	Implemented scope
Spending analysis	Monthly debit totals, category breakdowns and explainable anomaly flags.
Smart transfers	Named verified recipients, amount clarification, corrections and explicit approval; simulated MFA when required.
Subscription management	Recurring-charge candidates, stored-record eligibility and confirmed simulated cancellation.

This document explains the current implementation rather than claiming every example in the challenge brief is complete. Sections 2-3 describe the system; Sections 4-7 explain the algorithms; Sections 8-9 cover security; Sections 10-11 cover persistence, deployment and verification.

Design invariant: understanding a request, proposing an action, approving it and confirming its completion are separate steps.

2 System architecture

The FastAPI backend is the composition root. It is the only application layer that coordinates database access and financial execution. The agent, finance, policy and subscription packages expose typed interfaces and can be tested independently against shared contracts.



Layer	Responsibility
React and TypeScript	Chat, saved conversations, account views, transaction views, subscription views and approval or MFA controls.
Web serving and FastAPI	Locally, Nginx serves the frontend and proxies `/api/v1`. On Render, the FastAPI shared gateway serves the compiled frontend and protects API access. The backend coordinates controlled operations.
Agent controller	Intent extraction, input grounding, clarification state, bounded planning and user-facing replies.
Finance module	Exact spending aggregation, categorization and rule-based anomaly detection.
Security policy module	Pure fail-closed policy decisions; the backend persists and enforces their outcomes.
Subscription module	Pure recurrence detection, candidate mapping and cancellation eligibility.
SQLAlchemy and PostgreSQL	Accounts, ledger entries, stored subscriptions, approvals, audits, conversation state and transcripts.

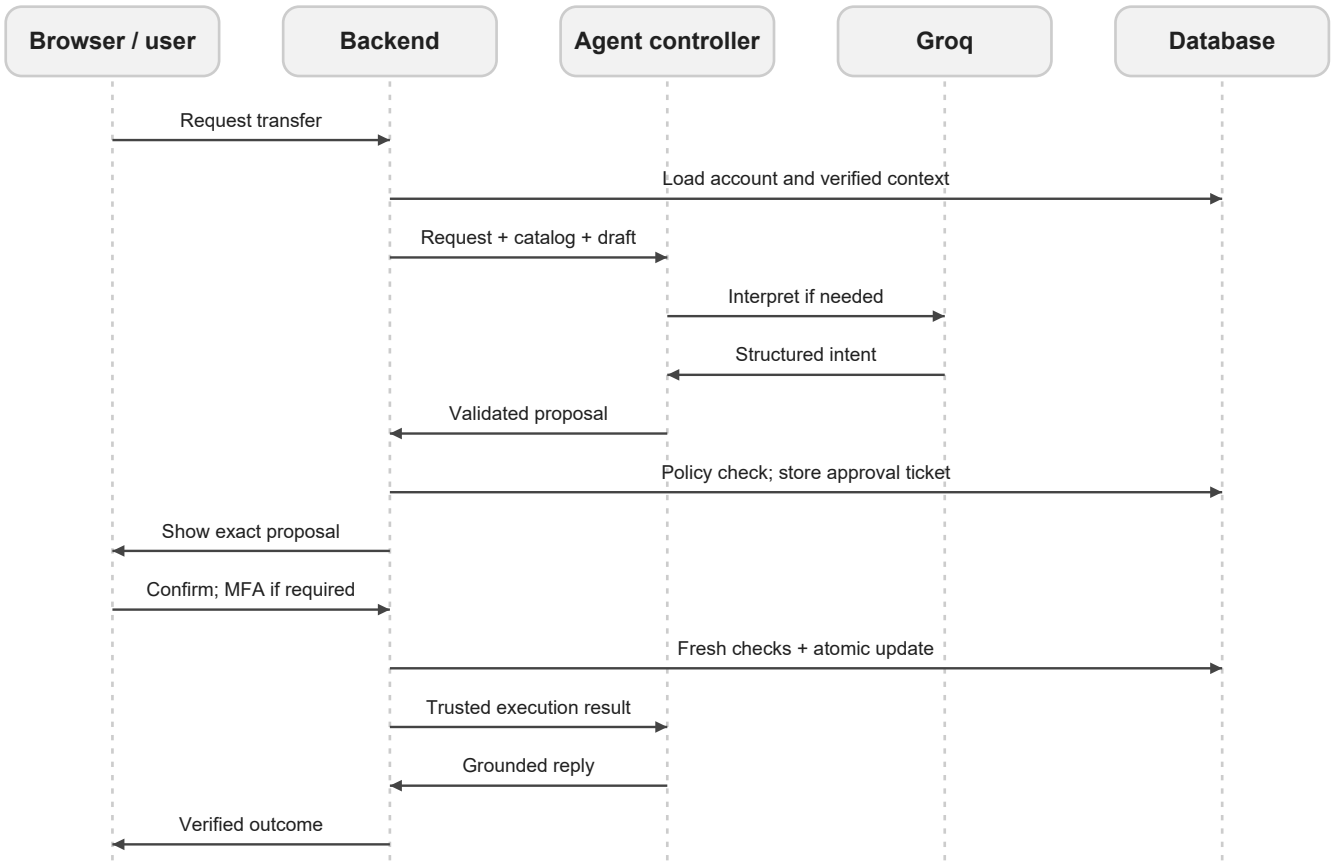
The external model receives the request text, instructions and output schema. Account selection, recipient IDs, subscription state and tool-result validation are handled in application code. Financial reports are formatted locally rather than sent to an LLM to calculate or invent values.

Local Docker Compose runs separate frontend, backend and PostgreSQL services. The hosted demo runs the compiled frontend and FastAPI together in one Render Docker service, with persistent PostgreSQL on Neon and Groq for interpretation. Render provides HTTPS; private environment variables hold database and provider credentials. SQLite supports local development and disposable tests. Independent mock adapters support module development, while the real agent also has a deterministic mock *interpretation provider*.

Implementation: `backend/app/services/adapters.py`, `backend/app/api/chat.py`, `frontend/src/App.tsx`, `docker-compose.yml`, `infra/`.

Request lifecycle: an approved transfer

The following sequence connects the architecture to one user journey: "Transfer 50 CNY to Bob." The backend resolves the caller's account and verified recipient context. The agent uses language interpretation and validation to prepare a proposal; the backend owns approval and execution.



Before approval: the balance is unchanged. The ticket binds the exact proposed tool and financial arguments; a new request does not silently authorize an old ticket.

At execution: the backend checks ownership, safety locks, current funds and the cumulative daily transfer threshold again. If the cumulative total exceeds CNY 1,000, strong verification is simulated before execution. Repeating confirmation for the same completed ticket does not debit it again.

After execution: the backend supplies a trusted, correlated outcome. The agent reports completion only from that result. A timeout during interpretation produces no new proposal; an execution failure rolls back partial financial writes.

Recognized simple requests may use local interpretation shortcuts, so the model exchange is conditional. For a balance or spending query, there is no approval step: the backend executes the permitted read and returns its result for a grounded reply. All illustrated financial effects occur in the synthetic simulator.

Implementation: `backend/app/api/chat.py`, `backend/app/api/approvals.py`, `backend/app/api/continuation.py`, `backend/app/services/action_execution.py`, `agent/service.py`.

3 Shared contracts and controlled tools

Shared Pydantic v2 models define the module boundaries. Unexpected fields are forbidden, IDs are bounded strings and transaction timestamps must include a timezone. Money uses Python `Decimal`; canonical monetary JSON values are strings. Positive transfer amounts have at most two decimal places. Versioned executable contracts take precedence over preliminary interface examples.

Tool	Required arguments	Execution category
`get_balance`	`account_id`	Read only
`analyze_spending`	`account_id`, `period` as `YYYY-MM`	Read only
`list_subscriptions`	`account_id`	Read only
`request_transfer`	`source_account_id`, `recipient_id`, `amount`, `currency`	Approval required
`request_subscription_cancel`	`subscription_id`	Approval required

The backend constructs catalog entries shaped as `{name, parameters, context}`. Argument schemas come from `TOOL_ARGUMENT_MODELS`; trusted context supplies the selected account, verified recipients, stored subscriptions and conversation state. This is an application catalog, not a grant of native SDK tool-execution authority to the model.

The public agent interface is:

```
process_message(chat: ChatInput, tool_catalog: list[dict],
                tool_results: list[dict] | None = None) -> AgentResult
```

`AgentResult` contains `reply`, `proposals` and `needs_clarification`. The additional `process_turn` helper returns the result plus caller-owned state. A transfer proposal contains only canonical financial arguments:

```
{
  "proposal_id": "proposal_example",
  "tool": "request_transfer",
  "arguments": {
    "source_account_id": "acc_cny_001",
    "recipient_id": "usr_bob_01",
    "amount": "300.00",
    "currency": "CNY"
  }
}
```

The server separately issues an `action_id`, digest, tier and current status under sibling approval metadata. These fields are not injected into canonical tool arguments. `proposal_id` identifies the agent proposal; `action_id` identifies the stored approval ticket.

Implementation: `contracts/models.py`, `contracts/tool_schemas.py`, `contracts/enums.py`, `backend/app/api/schemas.py`.

4 Intent understanding and conversation recovery

Interpretation pipeline

1. Validate the request and the caller-supplied conversation state, including its conversation ID. Reject inconsistent context snapshots.

2. Process trusted tool results before ordinary language interpretation. Handle supported plan controls and whole-message draft abandonment locally.

3. Use deterministic shortcuts for recognized simple messages and exact clarification answers. Other requests go through the configured provider.

4. Validate the provider JSON against a private `Intent` model. Supported intents are balance, spending, subscriptions, transfer, cancel and unknown.

5. Ground extracted amounts and names in the user's request. Resolve names only against verified context; never silently repair an identity or invent a missing value.

6. Either ask for missing information or construct a schema-validated proposal. Store its original canonical snapshot for later result correlation.

The LLM handles broader phrasing and ordinary command typos. Conservative request guards distinguish direct mutation requests from quoted, hypothetical, negated or merchant-reported instructions. Typo normalization applies to request wording, not recipient identities, amounts or dates. Unknown input prompts clarification without proposing an operation.

Working memory and corrections

An unfinished transfer retains its original request, account, amount and recipient details. Spending drafts retain supplied month and year separately. A named month without a year asks for the year; a year alone asks for the month. Application parsing uses stated dates and ignores a model-invented year.

```
User: Transfer to vov
Agent: Asks for an amount
User: 300
Agent: Cannot find vov in the verified recipient list
User: Oh wait I meant Bob
Agent: Proposes CNY 300.00 to the uniquely matched verified Bob
```

Explicit corrections such as "actually, 50 instead" update an unfinished draft and preserve unrelated details. Invalid replacement amounts do not silently reuse an old amount. Natural stopping phrases discard a draft; existing approval cards remain unchanged. Once issued, a proposal cannot be rewritten through a draft correction.

Provider behavior

Groq is the live interpretation provider; the deterministic mock remains available for offline development. Both use the same validated intent boundary. Groq requests strict JSON schema output and low reasoning effort. Connections are reused per worker thread; request timeouts are bounded and automatic provider retries or fallback are not enabled. Provider failure returns a categorized safe reply and no new proposal. Tone cues affect wording only, never authorization.

Implementation: `agent/service.py`, `agent/request_guard.py`, `agent/conversation.py`, `agent/provider.py`, `agent/prompts.py`, provider adapters.

5 Grounded replies and ordered task planning

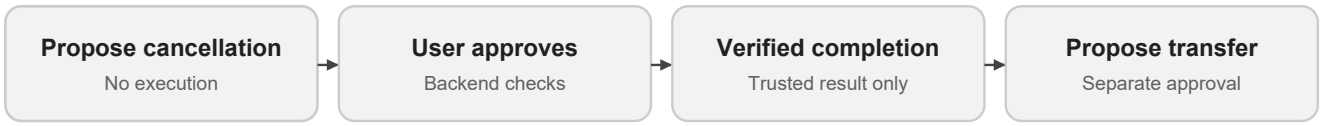
Read tool round trip

For a balance, spending or subscription query, the agent first proposes a read tool. The backend validates policy and account scope, runs the database query or deterministic calculation, and supplies a result envelope back to the agent. The envelope includes `conversation_id`, `proposal_id`, `tool`, `arguments`, `status` and typed `data`.

The agent requires the conversation, proposal, tool and canonical arguments to match the saved pending request. A balance must refer to the requested account; a report must refer to the requested month. Uncorrelated, altered or malformed results cannot establish success. Accepted read results consume the pending snapshot. Accepted terminal execution results are consumed; pending confirmation or MFA outcomes retain it for a later update.

Bounded dependency graph

The current planner supports two ordered nodes with one dependency edge. It is implemented as an explicit state machine, not arbitrary LLM-generated plans.



For example, "Cancel iQIYI VIP Monthly then transfer 45 CNY to Bob" starts with cancellation only. The transfer is proposed only after the backend verifies cancellation completion. The amount and recipient must be supplied explicitly; cancellation does not credit money to the account or establish that savings have already been received. Each action needs its own approval and fresh policy check.

Event	Planner behavior
Missing details	Retain the current step and ask for clarification.
Pending confirmation or MFA	Wait for a trusted update; do not release the dependent step.
Completed cancellation	Propose the specified transfer, subject to validation and approval.
Failed, rejected or cancelled action	Stop dependent work.
Pause or human takeover wording	Pause the plan and direct the user to the app team; no automatic human assignment occurs.
Stop plan	Remove unfinished steps, preserve issued approvals and do not reverse completed actions.

Continuation responses are cached against stored actions. Missed callbacks can be recovered on a later chat request. This is execution-result recovery, not permission to repeat a financial operation.

Implementation: `agent/planning.py`, `agent/service.py`, `backend/app/api/chat.py`, `backend/app/api/continuation.py`.

6 Spending analysis algorithm

The finance module is a deterministic pure calculation. The backend passes the selected account's transaction records; the finance function performs no database writes, HTTP calls or model inference. Months use Asia/Shanghai calendar boundaries, including when stored UTC timestamps fall near midnight.

Cleaning and aggregation

Identical repeated transaction IDs are counted once. Conflicting records sharing an ID are excluded and flagged. Defensive amount and timestamp checks exclude malformed records. Existing informative category labels are preserved, normalized and grouped case-insensitively; missing categories use ordered merchant rules, then `Uncategorized`.

```
Spending(month) = sum(valid debit amounts in that month)
CategoryTotal(category) = sum(valid debits in that category)
Spending(month) = sum(all category totals)
```

Credits, including income, inbound transfers and refund credits, are excluded. Refunds do not reduce gross spending. Outbound transfers count as spending by default. Decimal arithmetic and cent formatting are used throughout; financial totals do not depend on language-model arithmetic.

Explainable anomaly rules

Rule	Default trigger
Unusually large debit	With at least 3 category-history debits, amount is greater than 3 times their median and at least CNY 100 above it.
Large debit with little history	With insufficient category history, amount is at least CNY 2,000.
Category monthly spike	With the required 3-month history coverage and at least 2 active prior months, current total exceeds 1.5 times the prior monthly mean and is at least CNY 200 above it.
Possible duplicate charge	Same account, normalized merchant and amount within 600 seconds, with distinct transaction IDs. Both charges remain counted.

Each returned anomaly refers to a transaction ID and a stable reason code with an explanation. Flags are review signals, not proof of fraud. Richer informational diagnostics, such as excluded credits, are available separately and are not automatically fields in the shared report DTO.

The frozen October 2026 fixture produces total spending of **CNY 1,843.00**, with Shopping 1,200.00, Food 360.00, Transport 125.00, Education 80.00, Subscriptions 58.00 and Other 20.00; it has no anomaly flags. This fixture result must not be confused with the mutable application's seeded database, whose reports change after demo transfers.

The agent formats validated report values and known reason codes locally. Untrusted free-text anomaly instructions are not forwarded as commands or used to invent merchant details.

Implementation: `finance/analytics.py`, `finance/cleaning.py`, `finance/categories.py`, `finance/anomalies.py`, `finance/money.py`.

7 Recurring payments and cancellation eligibility

The subscription detector operates on the selected account's debits. It removes duplicate IDs and identical posted charges, groups normalized merchant names, clusters similar amounts and checks the gaps between consecutive charges. Transaction notes are not read by the detector.

Detection rules

At least two charges are needed. Amounts in one cluster must stay within 1.05 times its minimum. Every consecutive gap in that cluster must fall within one supported cadence window. The estimated payment is the median amount, rounded to cents.

Cadence	Consecutive gap window
Weekly	6 to 8 days
Monthly	27 to 33 days
Annual	360 to 370 days

Confidence is a rule-based evidence score: 0.40 for two matching charges, 0.70 for three, 0.85 for four and 0.90 for five or more. Varying amounts reduce it by 0.10. This score is not a calibrated probability or authorization decision. A candidate includes its evidence transaction IDs, cadence and estimated amount. The detector itself always sets cancellation support to false because transactions cannot establish merchant support.

Stored records govern cancellation

Candidate mapping first restricts stored subscriptions to the resolved user. A matching ID must also have a matching normalized merchant; otherwise merchant matching is used, with price disambiguation when needed. Missing or ambiguous matches are refused. The stored record must be active and support cancellation. Candidate confidence never bypasses these checks.

Example stage	Synthetic iQIYI example
Evidence	Two CNY 45.00 charges, 30 days apart.
Detected candidate	Monthly, estimated CNY 45.00, confidence 0.40.
Verified stored record	`sub_iQIYI_01`, active and cancellation supported.
Proposed action	Uses the stored ID and requires explicit confirmation.
Successful simulator execution	Marks the owned stored record cancelled and writes an audit; balance is unchanged.

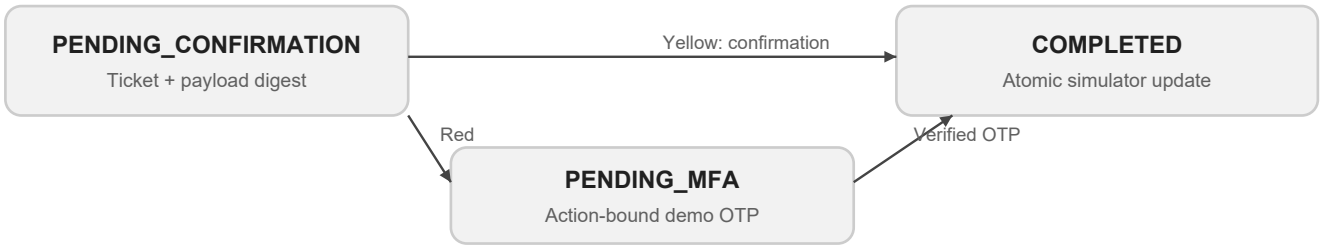
Detection candidates and verified subscriptions are deliberately separate. A user may also request cancellation of an eligible stored subscription without sufficient recurring evidence. Repeated cancellation cannot re-execute an already-cancelled record. No merchant API is contacted, and no refund or immediate savings credit is generated. Renewal reminders and scheduled notifications are not implemented.

Implementation: `subscriptions/detector.py`, `subscriptions/mapping.py`, `subscriptions/eligibility.py`, `backend/app/services/subscriptions.py`, `backend/app/services/executor.py`.

8 Permission tiers and execution safety

The policy engine is a pure, fail-closed function over a typed proposal and backend-resolved context. Invalid objects, unknown tools, invalid arguments, account mismatch and locked-account mutations are denied. A RED classification can therefore mean either an allowed action requiring MFA or a denied action; `allowed_to_proceed` must always be checked.

Tier	Current tool scope	Required user interaction
GREEN	Balance, spending and recurring-payment reads	Automatic after validation and account scoping.
YELLOW	Subscription cancellation; transfers where completed daily total plus requested amount is at most CNY 1,000	Explicit confirmation of the exact proposal.
RED	Transfers where that cumulative total exceeds CNY 1,000	Confirmation followed by simulated MFA.



Payload binding and fresh authorization

The server stores canonical financial arguments and computes SHA-256 over the tool and sorted payload JSON. Confirmation must refer to the owned ticket and its stored digest. The digest binds the approved payload; it is not a digital signature or a substitute for authentication. Arguments and approval metadata remain separate in HTTP responses.

At execution time the backend revalidates arguments, ownership, lock state, subscription eligibility and current daily transfer totals. A stale YELLOW request can escalate to RED after another completed transfer. A previously RED ticket is not downgraded. The daily counter uses Asia/Shanghai dates and rolls forward under the execution guard.

Concurrency and idempotent replay

The shared guard acquires database protection in user, action and account order. PostgreSQL row protection and SQLite writer serialization keep fresh policy checks, sufficient-funds checks, balance updates, ledger writes, audit writes and terminal status in the controlled transaction. Execution failures roll back partial financial writes.

Repeated confirmation or verification of a completed ticket returns its stored outcome without executing it again. This guarantee is tied to the server-issued action and its state; it is not a global deduplication guarantee across independently created tickets or arbitrary client keys.

The transfer simulator debits the source account and creates an outgoing ledger record. It does not settle funds to a recipient's real bank account. Stopping a draft is different from revoking an approval, and neither reverses a completed transfer.

Implementation: `security/policy.py`, `security/rules.py`, `backend/app/services/actions.py`, `action_execution.py`, `daily_transfers.py`, `executor.py`, `backend/app/api/approvals.py`.

9 MFA safety locks and threat controls

Simulated strong verification

RED-tier confirmation issues a random six-digit code bound to the action record and its owner. The challenge lasts up to five minutes and cannot outlive the ten-minute approval ticket. Three failed attempts are allowed. Expired, consumed or unissued challenges cannot authorize execution. Successful verification claims the challenge conditionally within the guarded execution transaction.

Delivery is explicitly simulated: the backend console and the resolved user's demo inbox display the code. No SMS or email is sent. Because the same demo interface exposes the code, this is not an independent production authentication factor. Plaintext OTP storage and console delivery must be replaced before real use.

Persistent safety locks

Three wrong OTPs or tampered confirmation digests within a fifteen-minute observation window trigger a user safety lock. Other pending approvals are retired, transfers and cancellations are blocked, and reads remain available. The lock survives restarts and does not expire automatically. A local operator reset requires a review reason and writes an audit; there is no agent-accessible unlock tool.

Threat	Implemented control
Invented banking results	Match tool results to saved proposals, canonical arguments, account and month before displaying figures or completion.
Prompt injection	Treat merchant and note text as data; validate output; whitelist tools; suppress suspicious display text; never execute generated code.
Changed payload or wrong account	Resolve demo identity server-side, scope records to that identity, validate exact arguments and compare the stored digest.
Repeated or racing execution	Database guard, conditional challenge consumption, terminal-state checks and atomic financial writes.
Provider outage or malformed output	Categorized failure responses, bounded requests and no new proposal on failure.
Repeated verification abuse	Challenge attempt limits, cross-action failure counting and persistent safety locks.

Audit records cover execution, MFA issuance, escalation, observed action status, policy rejections, daily-counter resets, approval cancellation and safety-lock events. The user-facing audit API exposes event metadata rather than raw internal details. Records use ordinary database storage; the prototype does not provide cryptographic tamper-evident logs or comprehensive capture of every model decision and read.

Provider error logs use a random event ID, stage, safe category and optional HTTP code. This redaction does not extend to the deliberately simulated OTP console or all legacy backend and policy logging paths; those require hardening before processing sensitive data.

Implementation: `backend/app/services/mfa.py`, `safety_lock.py`, `backend/app/api/approvals.py`, `demo_inbox.py`, `audit.py`, `agent/service.py`.

10 Persistence history and API design

SQLAlchemy models separate financial state, authorization records, working memory and the visible transcript. The default Docker deployment stores PostgreSQL data in the persistent `pgdata` volume. Alembic migrations are applied before the seed routine starts the backend; existing balances and ledger data are preserved.

Database table	Stored responsibility
`users` and `accounts`	Demo identities, balances, transfer-day counters and safety-lock state.
`transactions`	Debit or credit ledger records and their categories.
`subscriptions`	Owned stored subscriptions, status and cancellation support.
`action_approvals`	Canonical payload, digest, tier, status, expiry, MFA lifecycle and continuation correlation.
`audit_logs`	Action and security event records with audit IDs.
`agent_conversations`	User-scoped conversation ID, title, timestamps and agent working-state JSON.
`chat_messages`	Ordered user and agent messages, saved response data and unique continuation-event keys.

Working memory is distinct from visible history. The backend restores the selected conversation's draft state between turns. The frontend loads messages from server endpoints and stores only the selected conversation ID in browser storage. Full historic transcripts are not automatically sent as model context.

History reads restore approval cards from owned server tickets, verify their original arguments and digest, and use current ticket status. They do not invoke the model, create proposals, confirm approvals or execute operations. Continuation event keys prevent duplicate stored follow-up messages on repeated delivery. Message history is paginated and separate conversations do not share draft details.

API group	Current endpoints under `/api/v1`
Conversation	`POST /chat`, `POST /conversations`, `GET /conversations`, `GET /conversations/{id}`
Account data	`GET /accounts`, `GET /transactions`, `GET /subscriptions`, `GET /subscriptions/records`
Approval lifecycle	`GET /actions/{id}`; `POST /actions/{id}/confirm`, `/verify`, `/cancel`, `/continue`
Operational views	`GET /audit`, `GET /demo/otp-inbox`, `GET /health`

The local identity mechanism uses a demo header with a configured fallback user, not individual customer authentication; a caller can select another known demo identity. The hosted shared gateway adds HTTP Basic access control over HTTPS, pins requests to synthetic user `usr_demo_01` and ignores caller-selected user headers. All authorized visitors share that account, saved conversations, balances, subscriptions, daily counters and safety locks. This shared gate is not a personal banking login. Chat content, approvals and audit details are stored without application-layer encryption; retention and deletion controls are not yet implemented. Earlier messages that were never stored cannot be recovered retroactively.

Implementation: `backend/app/db/models.py`, `backend/migrations/`, `session_store.py`, `chat_history.py`, `backend/app/api/conversations.py`, `frontend/src/App.tsx`, `frontend/src/api.ts`.

11 Deployment and verification

Shared demonstration

The shared website is bankly-demo.onrender.com. Render runs the compiled frontend and FastAPI in one Docker web service. Neon stores PostgreSQL data independently of the service filesystem. Groq provides interpretation using `openai/gpt-oss-20b`.

The demo username is `bankly`; the password is supplied privately to reviewers. Render stores the password, Groq API key and database connection string as environment variables. These secrets are not included in the technical document or repository. The shared gateway protects website and API access and pins requests to the synthetic demo account.

All authorized visitors share account balances, transactions, subscriptions, saved conversations and safety locks. Restarting the service preserves Neon data. Startup applies migrations and seeds an empty database without resetting an existing one. The free service may sleep when idle, so the first visit can take longer to load. All banking effects remain simulated.

Start with the repository's README.md, which separates three options: try the hosted demo, run locally, or deploy a shared instance. Local Docker and development instructions are in docs/app/deployment.md. Render + Neon + Groq hosting instructions are in docs/app/free_shared_deployment.md. The two guides cover different environments. Local Docker uses its own database rather than automatically sharing the hosted Neon data.

Verification approach

Automated checks cover agent interpretation and conversation state, shared contracts, finance calculations, subscription detection, approval and MFA lifecycles, execution safety, persistence, and frontend behavior. The hosted gateway has additional checks for password protection, fixed demo identity, origin restrictions and request limits.

Backend and domain tests are under `backend/tests/`, `agent/tests/`, `finance/tests/`, `security/tests/`, `subscriptions/tests/` and `tests/test_contracts.py`. Frontend checks run with `npm test` and `npm run build` from `frontend/`. Gateway checks are in `infra/test_shared_app.py`. PostgreSQL-specific checks require a separately configured disposable test database.

Test totals depend on the tested repository revision; earlier totals are omitted here rather than presented as current results. No new application test run is claimed by this document update. Automated provider tests use synthetic responses and do not establish live model accuracy or network reliability.

For a judging rehearsal, verify a balance read, a spending clarification, a transfer proposal with no balance change before approval, the approved balance change, subscription cancellation and persistence after refresh. Check required MFA and plan continuation separately. Coordinate testing because all shared visitors affect the same synthetic account.

Implementation: `render.yaml`, `infra/Dockerfile.shared`, `infra/start_shared.py`, `infra/shared_app.py`, `backend/app/db/seed.py`.